

Project Title
A Dictionary-Based Fuzzy Search and Spell Correction Engine for Smart Shopping

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Bhanu Jakka (2520030006)
Akshita Anand (2520030043)
Harini P (2520030177)

Team Number: 11
Section: 7

Under the guidance of

Dr S Madhavi
Professor, CSE Department



Hyderabad-500090, Telangana, India.

ABSTRACT

Online shopping platforms rely heavily on search boxes to help customers find products, but customers frequently misspell item names while typing, causing traditional exact-match search systems to return no results. This problem grows more common as product catalogs expand across diverse categories, since customers rarely remember exact product names or spellings.

This project implements "**A Dictionary-Based Fuzzy Search and Spell Correction Engine for Smart Shopping**", a console-based smart shopping system that retrieves and corrects misspelled product searches in real time. The product catalog is indexed using a **Trie** data structure, allowing efficient prefix-based storage and retrieval of product names. When a customer types a misspelled item name, the system uses the **Levenshtein Edit Distance algorithm**, computed via dynamic programming while traversing the Trie, to identify and suggest the closest matching product — displaying a "Did you mean...?" correction along with full item details (category, price, and description).

Beyond spell correction, the system supports several complementary search and browsing features that reflect real-world shopping needs: **autocomplete** suggestions as a customer types a partial product name (using Trie prefix traversal), **keyword-based substring search** to find products containing a given term (using the **Rabin-Karp** rolling-hash algorithm), **category-wise browsing** of the catalog, **price-range filtering** using **Binary Search** over a price-sorted product list, and a **Top-N cheapest/most expensive** item finder implemented with a **PriorityQueue (heap)**. A simple **shopping cart**, backed by a hash map, allows customers to add items and view a running total, while a search-history tracker records trending searches using frequency counting.

Together, these components form a complete, menu-driven shopping search engine that demonstrates multiple core data structures and algorithms — Tries, dynamic programming, hashing, binary search, and heaps — applied cohesively to solve the practical problem of typo-tolerant, efficient product discovery in an e-commerce setting.